Cheatsheet AZ-104 — 07 · ARM templates y Bicep: estructura, despliegue, exportación y conversión

1. Anatomía de una plantilla ARM (JSON)

Sección	Obligatoria	Qué contiene
`$schema`	Sí	URL del esquema según el ámbito: `deploymentTemplate.json#` (resource group), `subscriptionDeploymentTemplate.json#`, `managementGroupDeploymentTemplate.json#`, `tenantDeploymentTemplate.json#`
`contentVersion`	Sí	Versión de la plantilla (`1.0.0.0`); solo informativa
`parameters`	No	Valores que se piden al desplegar: `type` (string, int, bool, object, array, **secureString**, secureObject), `defaultValue`, `allowedValues`, `minLength`, `maxLength`, `minValue`, `maxValue`, `metadata.description`
`variables`	No	Valores calculados reutilizables
`functions`	No	Funciones definidas por el usuario (raras)
`resources`	Sí	Lista de recursos: `type`, `apiVersion`, `name`, `location`, `properties`, `dependsOn`, `condition`, `copy`, `tags`, `sku`, `kind`
`outputs`	No	Valores devueltos al terminar (IDs, FQDN, claves)

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "prefijo": { "type": "string", "minLength": 3, "maxLength": 11 },
    "location": { "type": "string", "defaultValue": "[resourceGroup().location]" },
    "sku": { "type": "string", "defaultValue": "Standard_LRS", "allowedValues": [ "Standard_LRS", "Standard_GRS" ] }
  },
  "variables": { "stName": "[toLower(concat(parameters('prefijo'), uniqueString(resourceGroup().id)))]" },
  "resources": [ {
    "type": "Microsoft.Storage/storageAccounts", "apiVersion": "2023-05-01",
    "name": "[variables('stName')]", "location": "[parameters('location')]",
    "sku": { "name": "[parameters('sku')]" }, "kind": "StorageV2",
    "properties": { "supportsHttpsTrafficOnly": true }
  } ],
  "outputs": { "stId": { "type": "string", "value": "[resourceId('Microsoft.Storage/storageAccounts', variables('stName'))]" } }
}
```

- Las **expresiones** van entre corchetes `[ ]`. Funciones que caen: `parameters()`, `variables()`, `resourceGroup().location`, `resourceGroup().id`, `subscription().subscriptionId`, `uniqueString()` (hash determinista para nombres únicos), `concat()` / `format()`, `toLower()`, `resourceId()`, `reference()` (propiedades en runtime de un recurso), `listKeys()` (claves de storage), `copyIndex()`, `utcNow()` (solo en defaultValue de un parámetro), `deployment().name`.
- **dependsOn** fija el orden cuando un recurso necesita a otro (NIC → VNet, VM → NIC); `reference()` y `listKeys()` **por nombre** a un recurso de la misma plantilla crean dependencia implícita; `resourceId()` no (añade `dependsOn`). **copy** despliega N instancias (`"copy": { "name": "vms", "count": 3 }` + `copyIndex()`); **condition** despliega solo si se cumple (`"condition": "[equals(parameters('crearPip'), 'yes')]"`).
- Secretos: parámetro `secureString` y en el **fichero de parámetros** una referencia a **Key Vault** (`"reference": { "keyVault": { "id": "..." }, "secretName": "adminPassword" }`); el vault necesita **"Enabled for template deployment"** y el usuario permiso `Microsoft.KeyVault/vaults/deploy/action`.

2. Bicep: lo mismo, más corto

```
targetScope = 'resourceGroup'                    // subscription | managementGroup | tenant

@minLength(3)
param prefijo string
param location string = resourceGroup().location
@allowed([ 'Standard_LRS', 'Standard_GRS' ])
param sku string = 'Standard_LRS'
@secure()
param adminPassword string
var stName = toLower('${prefijo}${uniqueString(resourceGroup().id)}')
resource st 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: stName
  location: location
  sku: { name: sku }
  kind: 'StorageV2'
  properties: { supportsHttpsTrafficOnly: true }
}

resource vnet 'Microsoft.Network/virtualNetworks@2024-01-01' existing = { name: 'vnet-prod' }   // referenciar sin crear
resource nic 'Microsoft.Network/networkInterfaces@2024-01-01' = [for i in range(0, 3): {
  name: 'nic-${i}'
  location: location
  properties: { ipConfigurations: [ { name: 'ipcfg', properties: { subnet: { id: vnet.properties.subnets[0].id } } } ] }
}]
module web './modules/webapp.bicep' = if (sku == 'Standard_GRS') { name: 'webDeploy', params: { location: location } }

output stId string = st.id
```

- **Sin dependsOn explícito**: Bicep deduce las dependencias por los **nombres simbólicos** (`st.id`, `vnet.properties...`). Tipos y versiones de API con autocompletado en VS Code (extensión Bicep).
- Palabras clave: `param`, `var`, `resource`, `module` (otro .bicep o un template spec / registry), `output`, `existing`, `targetScope`, decoradores `@secure()`, `@allowed()`, `@description()`, `@minLength`; bucles `[for x in lista: {...}]`, condiciones `= if (...) {...}`, interpolación `'${a}-${b}'`, operador `?:`.
- Herramientas: `az bicep install` / `az bicep upgrade` / `az bicep version` · `az bicep build --file main.bicep` (Bicep → JSON) · `az bicep decompile --file template.json` (JSON → Bicep; revisar el resultado) · `bicep build` / `bicep decompile` si tienes el CLI de Bicep. Parámetros en `main.bicepparam` (`using 'main.bicep'` + `param prefijo = 'contoso'`) o en el `.parameters.json` clásico.

3. Desplegar

Ámbito	Azure CLI	PowerShell	Para qué
Resource group	`az deployment group create -g RG1 --template-file main.bicep --parameters main.bicepparam` · `--parameters @main.parameters.json` · `--parameters prefijo=contoso`	`New-AzResourceGroupDeployment -ResourceGroupName RG1 -TemplateFile main.bicep -TemplateParameterFile main.parameters.json` · `-prefijo contoso`	Lo habitual: recursos dentro de un RG
Subscription	`az deployment sub create -l westeurope --template-file main.bicep`	`New-AzSubscriptionDeployment -Location westeurope -TemplateFile main.bicep` (alias `New-AzDeployment`)	Crear RGs, Policy, RBAC a nivel de suscripción
Management group	`az deployment mg create -m mg-prod -l westeurope --template-file main.bicep`	`New-AzManagementGroupDeployment -ManagementGroupId mg-prod -Location westeurope ...`	Policy / RBAC para muchas suscripciones
Tenant	`az deployment tenant create -l westeurope ...`	`New-AzTenantDeployment ...`	Management groups

Opción	Detalle
Modo Incremental (default)	Crea o actualiza lo que está en la plantilla; **no toca** lo que ya existe en el RG y no aparece
Modo Complete	Además **borra** los recursos del RG que **no** están en la plantilla (solo en ámbito RG; `--mode Complete` / `-Mode Complete`). "Ensure the RG contains only what the template defines" → Complete. Peligroso: valida antes con what-if
What-if	`az deployment group what-if ...` / `--confirm-with-what-if` · `New-AzResourceGroupDeployment -WhatIf` → lista Create / Modify / Delete / NoChange antes de aplicar
Validate	`az deployment group validate ...` · `Test-AzResourceGroupDeployment` → sintaxis y esquema, no permisos ni cuotas
Historial	Resource group → **Deployments**: inputs, outputs, plantilla, errores; se puede **Redeploy**. Máximo **800** por RG (se auto-borran al acercarse; un lock CanNotDelete lo impide y los despliegues fallan)
Permisos	Contributor en el ámbito (`Microsoft.Resources/deployments/write` + los recursos); para asignar roles o Policy desde la plantilla, Owner / UAA / Resource Policy Contributor
Despliegue desde el portal	"Deploy a custom template" → editar, cargar fichero o elegir **Quickstart template**; cualquier recurso tiene **Automation options** / "Download a template for automation" antes de crearlo

- El mismo despliegue (mismo nombre) se puede repetir: es **idempotente**. Nombres de despliegue repetidos sobrescriben la entrada del historial.

4. Exportar, reutilizar y organizar

Qué	Cómo	Ojo
Export template de un RG o recurso	RG → Export template · recurso → Export template · `az group export -n RG1` · `Export-AzResourceGroup -ResourceGroupName RG1 -Path .`	Refleja el **estado actual**, puede faltar alguna propiedad o incluir valores fijos: revisar y parametrizar. No exporta secretos
Plantilla de un despliegue	RG → Deployments → despliegue → **Template**	Es **exactamente** la plantilla y parámetros que se usaron
Convertir JSON → Bicep	`az bicep decompile --file template.json`	Mejor desde una plantilla limpia; avisa de lo que no puede convertir
Template specs	`az ts create --name tsStorage --version 1.0 -g RG1 -l westeurope --template-file main.bicep` · `New-AzTemplateSpec` · desplegar: `az deployment group create --template-spec <id>` · `New-AzResourceGroupDeployment -TemplateSpecId <id>`	Plantilla **versionada como recurso** de Azure, compartida con RBAC (Template Spec Reader) sin repositorio ni SAS
Deployment stacks	`az stack group create --name s1 -g RG1 --template-file main.bicep --action-on-unmanage deleteAll --deny-settings-mode denyDelete`	Gestionan el ciclo de vida de lo desplegado (borrar lo que sale de la plantilla, deny settings); sustituyen a **Blueprints** (retirados)
Nested / linked templates	Nested: plantilla embebida en `Microsoft.Resources/deployments`. Linked: `templateLink.uri` a una URL **accesible** (storage + SAS, GitHub) o `relativePath` con template specs	Para modularizar; en Bicep se usan **módulos**
Quickstart templates	Galería `azure/azure-quickstart-templates`	Punto de partida; se despliegan desde el portal con "Deploy to Azure"

5. Errores típicos y qué significan

Error	Causa	Solución
`InvalidTemplate`	Sintaxis, expresión mal escrita, parámetro sin valor	Validar / what-if; revisar corchetes y comillas
`ResourceNotFound` al desplegar	Falta `dependsOn` o referencia a algo que aún no existe	Añadir dependencia (en Bicep, usar el nombre simbólico)
`DeploymentQuotaExceeded`	800 despliegues en el historial	Borrar entradas antiguas (`az deployment group delete`) o quitar el lock que impide limpiarlas
`RequestDisallowedByPolicy`	Una Azure Policy bloquea el recurso (ubicación, SKU, tag)	Cumplir la política o pedir exención
`SkuNotAvailable` / `OperationNotAllowed` (cuota)	SKU no disponible en la región o cuota agotada	Otra región / tamaño o aumentar cuota
`StorageAccountAlreadyTaken` / `Conflict`	Nombre global repetido	`uniqueString()` en el nombre
`MissingSubscriptionRegistration`	Resource provider no registrado	`az provider register --namespace Microsoft.X`
`NoRegisteredProviderFound` / apiVersion inválida	`apiVersion` inexistente para el tipo	Usar una versión válida (`az provider show -n Microsoft.Compute --query "resourceTypes[?resourceType=='virtualMachines'].apiVersions"`)
Recursos borrados tras desplegar	Se usó **modo Complete**	Usar Incremental o incluir todo en la plantilla
Acceso denegado al secreto de Key Vault durante el despliegue	Vault sin "enabled for template deployment" o sin permiso deploy/action	Activar la opción y dar permiso

6. Comandos que caen

Tarea	PowerShell (Az.Resources)	Azure CLI
Desplegar en RG	`New-AzResourceGroupDeployment -ResourceGroupName RG1 -TemplateFile main.bicep -TemplateParameterFile main.parameters.json -Mode Incremental`	`az deployment group create -g RG1 --template-file main.bicep --parameters main.bicepparam --mode Incremental`
Parámetro inline y URI	`-TemplateUri https://.../azuredeploy.json -TemplateParameterObject @{ prefijo = "contoso" }`	`--template-uri https://.../azuredeploy.json --parameters prefijo=contoso`
What-if / validar	`New-AzResourceGroupDeployment ... -WhatIf` · `Test-AzResourceGroupDeployment ...`	`az deployment group what-if ...` · `az deployment group validate ...`
Suscripción	`New-AzSubscriptionDeployment -Location westeurope -TemplateFile main.bicep`	`az deployment sub create -l westeurope --template-file main.bicep`
Historial	`Get-AzResourceGroupDeployment -ResourceGroupName RG1` · `Get-AzResourceGroupDeploymentOperation` · `Remove-AzResourceGroupDeployment`	`az deployment group list -g RG1 -o table` · `az deployment operation group list` · `az deployment group delete`
Exportar	`Export-AzResourceGroup -ResourceGroupName RG1 -Path .` · `Save-AzResourceGroupDeploymentTemplate -ResourceGroupName RG1 -DeploymentName d1`	`az group export -n RG1 > rg.json` · `az deployment group export -g RG1 -n d1`
Bicep	`bicep build main.bicep` · `bicep decompile template.json`	`az bicep build --file main.bicep` · `az bicep decompile --file template.json` · `az bicep upgrade`
Template spec	`New-AzTemplateSpec -Name ts1 -Version 1.0 -ResourceGroupName RG1 -Location westeurope -TemplateFile main.bicep`	`az ts create --name ts1 --version 1.0 -g RG1 -l westeurope --template-file main.bicep`

7. Árbol de decisión

```
¿Qué pide el enunciado?
├─ Desplegar lo mismo varias veces sin cambios (idempotente) ..... ARM / Bicep en modo Incremental
├─ Que el RG quede exactamente como la plantilla ..... Modo Complete (solo RG)
├─ Ver qué cambiará antes de aplicar ..... What-if
├─ Reutilizar lo que ya existe como plantilla ..... Export template (RG / recurso) o Deployments → Template
├─ Convertir una plantilla JSON a Bicep ..... az bicep decompile
├─ Compartir plantillas versionadas con el equipo ..... Template specs (RBAC)
├─ Contraseña en la plantilla sin exponerla ..... secureString + referencia a Key Vault en parámetros
├─ Nombre único para una storage account ..... uniqueString(resourceGroup().id)
├─ Tres VMs iguales ..... copy (ARM) / for (Bicep)
├─ Recurso solo en producción ..... condition (ARM) / if (Bicep)
├─ Crear resource groups o policies desde la plantilla ..... Despliegue de ámbito subscription / management group
├─ "DeploymentQuotaExceeded" ..... Borrar historial (800) o quitar lock CanNotDelete
└─ Sustituto de Blueprints ..... Template specs + deployment stacks
```

8. Señales → respuesta

Enunciado	Respuesta
"deploy the template without affecting existing resources"	Incremental (default)
"resources not in the template must be deleted"	Complete
"preview changes before deploying"	`what-if` / `-WhatIf`
"template fails: resource not found"	Falta dependsOn (o referencia a recurso no creado)
"storage account name must be unique", "same template in several RGs"	`uniqueString()`
"password must not be stored in the template or parameter file"	secureString + Key Vault reference (enabled for template deployment)
"deploy N identical VMs"	copy / for loop
"VM deploys only when parameter is true"	condition / if
"reuse the exact template that created the resources"	RG → Deployments → Template
"export template is missing some properties"	Normal: el export refleja el estado y no todas las propiedades; usar la del despliegue o completarlo
"store templates in Azure and control access with RBAC"	Template specs
"convert Bicep to JSON for a tool that only reads ARM"	`az bicep build`
"deploy to a subscription scope", "create resource groups"	`az deployment sub create` / `New-AzSubscriptionDeployment`
"the deployment history is full"	Borrar despliegues antiguos (límite 800)
"a resource lock prevents deployments"	CanNotDelete en el RG impide limpiar el historial
"assign a tag / policy during deployment at subscription"	Despliegue a nivel de suscripción con `Microsoft.Authorization/policyAssignments`

9. Errores a evitar

1. Usar modo Complete en un RG compartido: borra todo lo que no esté en la plantilla.
2. Olvidar `dependsOn` en JSON (en Bicep lo resuelven los nombres simbólicos).
3. Poner contraseñas en texto claro: `secureString` + Key Vault.
4. Hardcodear la región: `resourceGroup().location` o un parámetro.
5. Esperar que `validate` detecte cuotas, políticas o nombres repetidos: solo comprueba esquema y sintaxis; `what-if` va más lejos.
6. Exportar un RG y desplegarlo tal cual sin parametrizar nombres y revisar propiedades.
7. Confundir template spec (plantilla versionada) con deployment stack (gestión del ciclo de vida) o con Blueprints (retirado).
8. Desplegar a nivel de RG una plantilla cuyo `$schema` es de suscripción (o al revés).